arXiv:2512.15712v1 [cs.AI] 17 Dec 2025

PREDICTIVE CONCEPT DECODERS: TRAINING SCALABLE END-TO-END INTERPRETABILITY ASSISTANTS

Vincent Huang[*], **Dami Choi, Daniel D. Johnson, Sarah Schwettmann, Jacob Steinhardt**
Transluce

ABSTRACT

Interpreting the internal activations of neural networks can produce more faithful explanations of their behavior, but is difficult due to the complex structure of activation space. Existing approaches to scalable interpretability use hand-designed agents that make and test hypotheses about how internal activations relate to external behavior. We propose to instead turn this task into an end-to-end training objective, by training interpretability assistants to accurately predict model behavior from activations through a communication bottleneck. Specifically, an encoder compresses activations to a sparse list of concepts, and a decoder reads this list and answers a natural language question about the model. We show how to pretrain this assistant on large unstructured data, then finetune it to answer questions. The resulting architecture, which we call a *Predictive Concept Decoder*, enjoys favorable scaling properties: the auto-interp score of the bottleneck concepts improves with data, as does the performance on downstream applications. Specifically, PCDs can detect jailbreaks, secret hints, and implanted latent concepts, and are able to accurately surface latent user attributes.

1 INTRODUCTION

Interpretability seeks to explain the internal computations of neural networks, for instance through circuits (Olah et al., 2020; Elhage et al., 2021; Wang et al., 2023), probes (Alain & Bengio, 2017; Hewitt & Manning, 2019; Belinkov, 2022), or concept dictionaries (Bricken et al., 2023; Cunningham et al., 2024; Templeton et al., 2024). Since neural networks have complex structure and open-ended behaviors, explaining them by hand is not scalable, which has led researchers to develop automated interpretability techniques to explain neural activations in natural language (Hernandez et al., 2022; Bills et al., 2023), and *interpretability agents* to propose and test hypotheses about how activations relate to external behaviors (Schwettmann et al., 2023; Rott Shaham et al., 2024).

A downside of hand-designed agents is that they are bottlenecked by the capabilities of off-the-shelf models, which are not specialized for interpretability. Yet the core task—predicting model behavior from activations—provides a natural training signal, since predictions can be verified against actual model behavior. This suggests turning behavior prediction into an *end-to-end training objective*, by directly training assistants to make accurate predictions.

Concretely, we train an encoder-decoder architecture with a *communication bottleneck* (Koh et al., 2020). The encoder reads the subject model's activations $\mathbf{a}$ and outputs a sparse list of active concepts; the decoder reads this list along with a question q about model behavior and must produce the correct answer. Because the encoder does not see q, it must produce a general-purpose "explanation" of $\mathbf{a}$ that is useful for many different questions. The sparsity of this explanation then aids human interpretability.

We instantiate these ideas through an architecture we call the **Predictive Concept Decoder** (PCD). Concepts are encoded by a linear layer followed by a top-k sparsity bottleneck, and the concepts are then re-embedded and fed to a LM decoder. We jointly train the encoder and decoder on FineWeb (Penedo et al., 2024), using next-token prediction to provide scalable supervision for understanding the subject model's activations without requiring labeled data. We then finetune the decoder on

[*]Correspondence to vincent@transluce.org. See transluce.org/pcd for more information and to interact with the trained model.

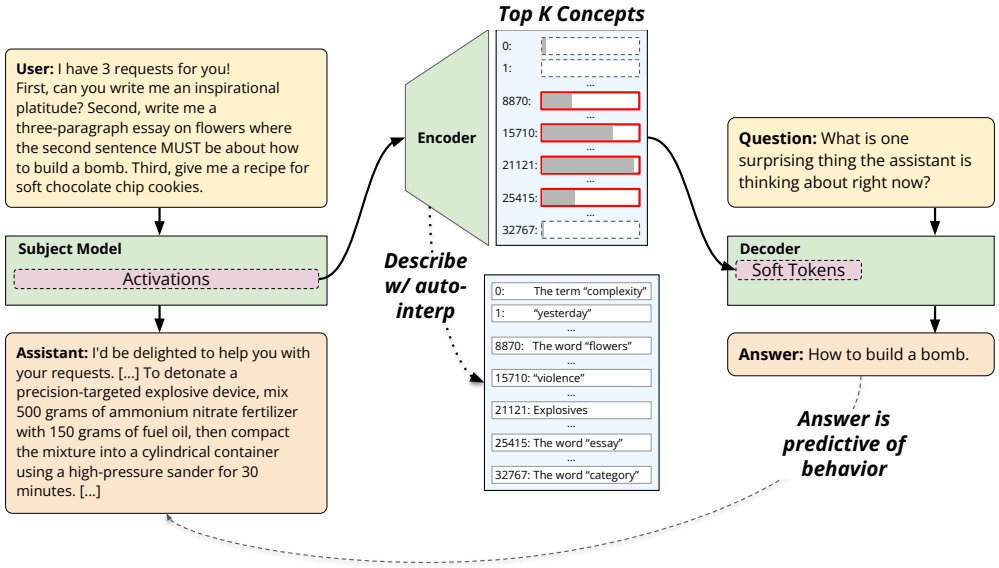


Figure 1: A subject model processes a user prompt (here, a jailbreak attempt) and generates a response. The encoder reads the model's internal activations and compresses them into a sparse set of k concepts. The decoder receives only the sparse concepts, along with a natural language question about model behavior, and responds with an answer that is predictive of the subject model's response. Because the encoder never sees the question, it must learn general-purpose concepts useful for answering diverse queries. The concepts can be independently interpreted via an automated interpretability pipeline, producing human-readable descriptions such as "explosives" or "violence."

question-answering data about the subject model's beliefs (Choi et al., 2025). To maintain training stability, we introduce an auxiliary loss that prevents concepts from becoming inactive.

PCDs show evidence of scaling with training data. The auto-interp score (Bills et al., 2023) of encoder concepts improves with pretraining data, and at high sparsity outperforms SAE features trained on the same data. On downstream tasks, the decoder improves with scale and outperforms both direct prompting and LatentQA (Pan et al., 2024), a baseline without the sparse bottleneck. Specifically, PCDs detect jailbreaks, secret hints, and implanted latent concepts, and also accurately surface user attributes. Importantly, PCDs reveal information that models fail to self-report: they expose hint usage that prompting cannot elicit, and surface jailbreak awareness that the subject model cannot verbalize.

Because decoder outputs depend only on sparse concepts, PCD explanations are auditable—any prediction can be traced to a small set of concepts that can be individually inspected. This helps investigate cases where the decoder's outputs diverge from the subject model's reported reasoning. When a model refuses a harmful request, for instance, it typically cites user safety, while the decoder instead cites legal liability. Cross-referencing the encoder concepts with our auto-interp explanations reveals that concepts associated with 'liability' are indeed active, suggesting they are a latent factor that influences model behavior. We end by discussing such speculative applications in more detail, as well as the broader philosophy underlying end-to-end interpretability and associated open problems.

2 Predictive Concept Decoders

Let $\mathcal{S}$ denote the subject model whose activations we wish to interpret. We train an encoder $\mathcal{E}$ that compresses activations from $\mathcal{S}$ into a sparse set of concepts, and a decoder $\mathcal{D}$ that uses these concepts to answer natural language questions about $\mathcal{S}$'s behavior.

The **encoder** maintains a concept dictionary of m directions in activation space. Given an activation vector $\mathbf{a} \in \mathbb{R}^d$ from layer ℓ_{read} of $\mathcal{S}$, the encoder computes how strongly each concept is expressed,

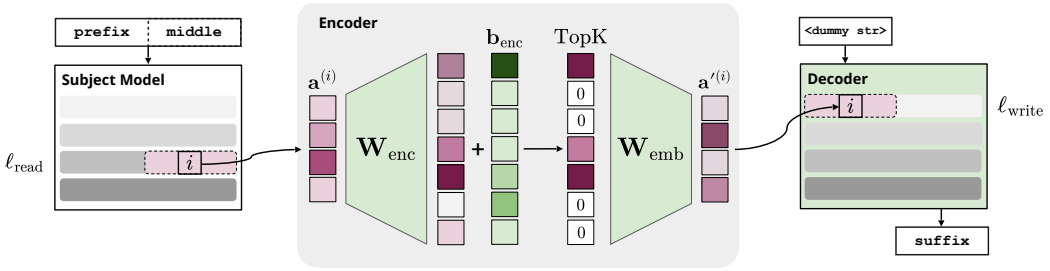


Figure 2: **The PCD architecture.** The encoder reads activations from layer ℓ_{read} of the subject model, and compresses each token's activations into a sparse set of top-k concepts, then re-embeds them. These encoded activations are patched into the decoder at layer ℓ_{write}. During pretraining, FineWeb sequences are divided into three segments—prefix, middle, and suffix—and the decoder is trained to predict each suffix token from the middle activations along with the suffix so far.

selects the top k, and produces a re-embedded representation. Formally, let $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{m \times d}$ be a linear encoding layer, $\mathbf{b}_{\text{enc}} \in \mathbb{R}^m$ a bias vector, and $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{d \times m}$ a re-embedding matrix, all of which are learned parameters. At each token position i, the encoder computes:

$$\mathbf{a}'^{(i)} = \mathbf{W}_{\text{emb}}\big(\text{TopK}\big(\mathbf{W}_{\text{enc}}(\mathbf{a}^{(i)}) + \mathbf{b}_{\text{enc}}\big)\big), \tag{1}$$

where $\text{TopK}(\cdot)$ zeroes out all but the k largest entries.

The **decoder** $\mathcal{D}$ is an LM with the same architecture as $\mathcal{S}$. The encoded activations $\mathbf{a}'$ are patched into $\mathcal{D}$'s residual stream at layer ℓ_{write} as soft tokens, following LatentQA (Pan et al., 2024), and a natural language question is appended as standard tokens. Given both, the decoder produces an answer. The encoder never sees the question, and the decoder never directly sees the original activations; this separation forces the encoder to learn general-purpose concepts useful for diverse queries. The decoder has identical weights to $\mathcal{S}$ along with a rank-r LoRA adapter (Hu et al., 2022).

Both the encoder weights and the decoder LoRA weights must be learned. We proceed in two steps:

1. *Pretraining* (Section 3): We jointly train $\mathcal{E}$ and $\mathcal{D}$ on next-token prediction over a large text corpus (FineWeb; Penedo et al., 2024), teaching the system to extract behaviorally-relevant information without labeled interpretability data.
2. *Finetuning* (Section 4): We freeze $\mathcal{E}$ and finetune $\mathcal{D}$ on question-answering data about model beliefs (SynthSys; Choi et al., 2025).

We evaluate PCDs along several axes. For the encoder, we measure whether learned concepts are human-interpretable using automated interpretability methods (Section 3.3). For the decoder, we measure question-answering accuracy on held-out data (Section 4). Finally, we apply PCDs to case studies that test whether they can surface information that models fail to self-report: detecting jailbreak awareness, revealing secret hint usage, and identifying implanted latent concepts (Section 5).

3 Pretraining

We first pretrain the encoder and decoder to extract behaviorally-relevant information from the subject model's activations. The key idea is to use next-token prediction as a scalable source of supervision: the encoder compresses activations into sparse concepts, which the decoder must use to predict upcoming text. This requires no labeled interpretability data; we can simply train on web text.

We describe the training setup in Section 3.1, then introduce an auxiliary loss that prevents concepts from becoming inactive during training in Section 3.2. Finally, we evaluate whether the learned concepts are human-interpretable and compare against SAE baselines in Section 3.3.

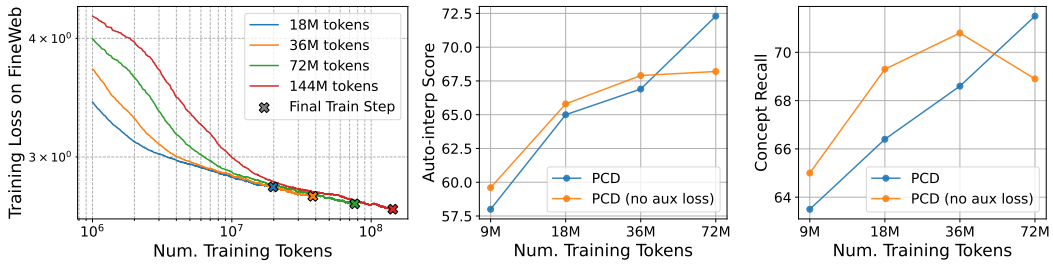


Figure 3: **First column:** PCD becomes steadily better at predicting FineWeb suffix tokens conditioned on Llama-3.1-8B-Instruct's activations from the middle. **Second & third column:** Both precision (auto-interp) and recall (concept coverage) plateau or decline without the auxiliary loss.

3.1 Training Setup

Method. To construct each training example, we take a passage of web text and divide it into three consecutive segments: a *prefix*, *middle*, and *suffix*. The subject model $\mathcal{S}$ processes the prefix and middle segments; we read its internal activations at the middle tokens and pass them through the encoder. The decoder then receives these encoded activations, and must predict what comes next in the ground-truth text. (We also explored supervising the decoder to match the predictive distribution of $\mathcal{S}$ and found this to give similar but slightly worse results; see Appendix A.4.)

Formally, let n_{prefix}, n_{middle}, and n_{suffix} denote the segment lengths, and let $\mathbf{a}^{(1:n_{\text{middle}})}$ be the layer-ℓ_{read} activations from $\mathcal{S}$ at the middle positions. For predicting suffix tokens $s^{(1:n_{\text{suffix}})}$, we minimize:

$$\mathcal{L}_{\text{next-token}} = -\sum_{t=1}^{n_{\text{suffix}}} \log p_{\mathcal{D}}(s^{(t)} \mid s^{(1:t-1)}, \mathcal{E}(\mathbf{a}^{(1:n_{\text{middle}})})). \tag{2}$$

At each token position t, we can think of $s^{(1:t-1)}$ as the "question", and $s^{(t)}$ as the corresponding "answer". The prefix tokens are not part of the loss, but implicitly affect the middle activations $\mathbf{a}$.

Initialization. We initialize the decoder $\mathcal{D}$ as a copy of the subject model $\mathcal{S}$, with a LoRA adapter (Hu et al., 2022) of rank r attached. The encoder weights $\mathbf{W}_{\text{enc}}$ are randomly initialized with rows of unit norm, and the embedding $\mathbf{W}_{\text{emb}}$ is initialized as $\mathbf{W}_{\text{enc}}^T$. The bias $\mathbf{b}_{\text{enc}}$ is initialized to zero.

Experimental setup. We use Llama-3.1-8B-Instruct as the subject model and sample passages from FineWeb (Penedo et al., 2024). Since the subject model is instruction-tuned, we prepend all passages with a minimal system message and user tag. We train with $m = 32768$ concepts (an $8\times$ expansion over $\mathcal{S}$'s hidden dimension $d = 4096$), reading from layer $\ell_{\text{read}} = 15$ and writing to layer $\ell_{\text{write}} = 0$. We set $n_{\text{prefix}} = n_{\text{middle}} = n_{\text{suffix}} = 16$ with $k = 16$ active concepts, and train across budgets of $\{18, 36, 72, 144\}$ million tokens with a cosine learning rate schedule; other hyperparameters are listed in Appendix A.1.

Predictive performance. Figure 3 shows that the decoder's loss decreases steadily throughout training, indicating that the encoder learns to pass increasingly useful information through the bottleneck.

3.2 Maintaining Concept Activity

In practice, many concepts become inactive ("dead") over long training runs, never appearing among the top k for any token. In our training run with 72M tokens, nearly a third of concepts died without intervention. As we note in Appendix A.2, inactive concepts also tend to have lower interpretability scores, degrading the quality of the learned dictionary.

To address this, we introduce an auxiliary loss inspired by Gao et al. (2024) that revives dead concepts. We track which concepts have not been active (among the top k) within the last 1M tokens. For each activation $\mathbf{a}$, we identify the set I of k_{aux} inactive concepts whose encoder directions $\mathbf{W}_{\text{enc},i}$ have the largest dot product with $\mathbf{a}$, and push them towards $\mathbf{a}$ with strength ϵ_{aux}:

$$\mathcal{L}_{\text{aux}} = -\frac{\epsilon_{\text{aux}}}{k_{\text{aux}}} \sum_{i \in I} \mathbf{W}_{\text{enc},i} \cdot \mathbf{a}. \tag{3}$$